

assumption is unlikely to be appropriate in all contexts. So it seems that generators must be told the size of test cases to generate. We can imagine an API where this is made explicit:

```
def listOfN[A](n: Int, a: Gen[A]): Gen[List[A]]
```

This would certainly be a useful combinator, but *not* having to explicitly specify sizes is powerful as well. It means that whatever function runs the tests has the freedom to choose test case sizes, which opens up the possibility of doing the test case minimization we mentioned earlier. If the sizes are always fixed and specified by the programmer, the test runner won't have this flexibility. Keep this concern in mind as we get further along in our design.

What about the rest of this example? The `forAll` function looks interesting. We can see that it accepts a `Gen[List[Int]]` and what looks to be a corresponding predicate, `List[Int] => Boolean`. But again, it doesn't seem like `forAll` should care about the types of the generator and the predicate, as long as they match up. We can express this with the type:

```
def forAll[A](a: Gen[A])(f: A => Boolean): Prop
```

Here, we've simply invented a new type, `Prop` (short for *property*, following the `ScalaCheck` naming), for the result of binding a `Gen` to a predicate. We might not know the internal representation of `Prop` or what other functions it supports, but based on this example we can see that it has an `&&` operator, so let's introduce that:

```
trait Prop { def &&(p: Prop): Prop }
```

8.2.2 The meaning and API of properties

Now that we have a few fragments of an API, let's discuss what we want our types and functions to *mean*. First, consider `Prop`. We know there exist functions `forAll` (for creating a property), `&&` (for composing properties), and `check` (for running a property). In `ScalaCheck`, this `check` method has a side effect of printing to the console. It's fine to expose this as a convenience function, but it's not a basis for composition. For instance, we couldn't implement `&&` for `Prop` if its representation were just the `check` method:³

```
trait Prop {
  def check: Unit
  def &&(p: Prop): Prop = ???
}
```

Since `check` has a side effect, the only option for implementing `&&` in this case would be to run both `check` methods. So if `check` prints out a test report, then we would get two of them, and they would print failures and successes independently of each other.

³ This might remind you of similar problems that we discussed in chapter 7, when we looked at using `Thread` and `Runnable` for parallelism.

That’s likely not a correct implementation. The problem is not so much that `check` has a side effect, but more generally that it throws away information.

In order to combine `Prop` values using combinators like `&&`, we need `check` (or whatever function “runs” properties) to return some meaningful value. What type should that value have? Well, let’s consider what sort of information we’d like to get out of checking our properties. At a minimum, we need to know whether the property succeeded or failed. This lets us implement `&&`.



EXERCISE 8.3

Assuming the following representation of `Prop`, implement `&&` as a method of `Prop`.

```
trait Prop { def check: Boolean }
```

In this representation, `Prop` is nothing more than a non-strict `Boolean`, and any of the usual `Boolean` functions (`AND`, `OR`, `NOT`, `XOR`, and so on) can be defined for `Prop`. But a `Boolean` alone is probably insufficient. If a property fails, we might want to know how many tests succeeded first, and what arguments produced the failure. And if a property succeeds, it would be useful to know how many tests it ran. Let’s try returning an `Either` to indicate success or failure:

```
object Prop {
  type SuccessCount = Int
  ...
}
trait Prop { def check: Either[???,SuccessCount] }
```

Type aliases like this can help the readability of an API.

What type shall we return in the failure case? We don’t know anything about the type of the test cases being generated. Should we add a type parameter to `Prop` and make it `Prop[A]`? Then `check` could return `Either[A,Int]`. Before going too far down this path, let’s ask ourselves whether we really care about the *type* of the value that caused the property to fail. We don’t really. We would only care about the type if we were going to do further computation with the failure. Most likely we’re just going to end up printing it to the screen for inspection by the person running the tests. After all, the goal here is to find bugs, and to indicate to someone what test cases trigger those bugs so they can go and fix them. As a general rule, we shouldn’t use `String` to represent data that we want to compute with. But for values that we’re just going to show to human beings, a `String` is absolutely appropriate. This suggests that we can get away with the following representation for `Prop`:

```
object Prop {
  type FailedCase = String
  type SuccessCount = Int
}
```